

RAG (Retrieval-Augmented Generation) - Sample Notes

What is RAG?

Retrieval-Augmented Generation, or RAG, is a technique that combines information retrieval with text generation. Instead of relying only on what a language model learned during training, RAG first retrieves relevant documents or text chunks from an external knowledge source, and then uses that retrieved information as context to generate a more accurate and up to date answer.

Why use RAG?

Large language models can only know what they were trained on, and their knowledge has a cutoff date. RAG solves this problem by letting the model look up fresh or private information, such as company documents, PDFs, or databases, at the time a question is asked. This reduces hallucination and makes the answers grounded in real data.

How RAG works, step by step

1. Load documents: Read the source data, for example PDFs, Word files, or web pages. 2. Chunk the text: Split long documents into smaller overlapping pieces so they fit into the model's context window. 3. Create embeddings: Convert each chunk into a numeric vector that represents its meaning. 4. Store in a vector index: Save the embeddings so they can be searched quickly later. 5. Retrieve: When a user asks a question, convert the question into an embedding too, then find the chunks with the closest matching embeddings. 6. Generate: Send the question along with the retrieved chunks to a language model, which then produces the final answer using that context.

Common tools used in RAG systems

Popular libraries and tools for building RAG pipelines include LangChain, LlamaIndex, FAISS, Chroma, and Pinecone for vector storage, along with embedding models from OpenAI, Cohere, or open source options like sentence-transformers. For simple projects, TF-IDF based similarity search can also work well without needing a dedicated vector database.

Example use case

A company can upload its internal policy documents into a RAG system. When an employee asks 'How many vacation days do I get?', the system retrieves the relevant paragraph from the HR policy PDF and passes it to the language model, which then answers accurately based on the actual company policy instead of guessing.